

Log-Structured File Systems

1 Basic Idea

Log-Structured File System

A **log-structured file system**, or **LFS**, treats the disk as one large log. Instead of writing small updates in many different disk locations, it collects pending writes in memory and writes them to disk in large contiguous segments.

- LFS was designed to improve file-system performance when disks have slow seeks.
- The main idea came from Berkeley research by Rosenblum and Ousterhout.
- It is especially focused on making small random writes more efficient.
- The disk is written sequentially, like appending records to a log.

2 Why LFS Was Needed

- CPUs keep getting faster.
- Main memory keeps getting larger.
- Disk capacity keeps growing.
- Disk seek time does not improve as quickly.
- Larger memory allows larger file-system caches.
- Many read requests can be served from cache without disk access.
- Therefore, many real disk accesses become writes rather than reads.

Small-Write Problem

Small disk writes are inefficient because a tiny write may still require a seek and rotational delay before the actual data transfer.

3 Cost of Small Writes

- A small disk write may take only about 50 microseconds of actual transfer time.
- It may be preceded by about 10 milliseconds of seek time.
- It may also wait about 4 milliseconds for rotational delay.
- The disk spends most of the time positioning, not transferring data.
- This can reduce disk efficiency to a fraction of 1 percent.

4 Why Ordinary File Systems Do Many Small Writes

Example: Creating a UNIX File

Creating a new file can require several separate writes, even before much user data is written.

- The directory i-node may need to be updated.
- The directory block must record the new file name.
- The new file's i-node must be written.
- The file's data block must be written.
- These writes affect different disk locations.
- Delaying them can create consistency problems if the system crashes.
- For this reason, important metadata writes are often done quickly.

5 Main LFS Mechanism

Write Strategy

LFS buffers many small writes in memory, collects them into a large segment, and writes that segment sequentially to the end of the log.

1. File-system updates are first buffered in memory.
2. Periodically, the buffered updates are grouped into one segment.
3. The segment is written contiguously at the end of the log.
4. The segment may contain file data, directory blocks, i-nodes, and other metadata.
5. A segment summary is placed at the start of the segment.
6. The segment summary describes what is stored inside the segment.

6 Segments

- A segment is a large contiguous unit written to disk.
- A segment may contain different kinds of blocks mixed together.
- Common segment contents include:
 - data blocks,
 - directory blocks,
 - i-nodes,
 - segment summary information.
- If segments are large enough, such as about 1 MB, the disk can be used close to full bandwidth.

7 I-Nodes in LFS

- LFS still uses i-nodes.
- The i-node structure can be similar to UNIX i-nodes.
- The difference is that i-nodes are not stored at fixed disk positions.
- They are scattered throughout the log.
- Once an i-node is found, its file-block addresses are used normally.

8 I-Node Map

I-Node Map

An **i-node map** is an index that maps an i-node number to the current disk location of that i-node.

- In traditional UNIX, an i-node's disk address can be calculated from its i-node number.
- In LFS, i-nodes move because new versions are appended to the log.
- Therefore, LFS needs an i-node map.
- Entry i in the map points to i-node i on disk.
- The map is stored on disk.
- Frequently used parts of the map are cached in memory.

9 Opening a File in LFS

1. The path name is searched to find the file's i-node number.
2. The i-node map is consulted.
3. The map gives the current disk address of the i-node.
4. The i-node is read.
5. The i-node gives the disk addresses of the file's data blocks.
6. The required file blocks are then read from their locations in the log.

10 Why Cleaning Is Needed

Finite Disk Problem

If the disk were infinite, LFS could keep appending forever. Real disks are finite, so old and obsolete log segments must eventually be reclaimed.

- When a file block is overwritten, the new version is appended to the log.
- The old version still occupies space in an older segment.
- The current i-node points to the new block, not the old block.
- The old block is now obsolete garbage.
- LFS needs a way to reclaim such space.

11 Cleaner Thread

Cleaner

The **cleaner** is a background thread that scans old log segments, keeps live data, discards obsolete data, and marks old segments free.

1. The cleaner scans the log circularly.
2. It reads the segment summary of an old segment.
3. It checks which i-nodes and blocks are described there.
4. It consults the current i-node map to see whether those items are still live.
5. Obsolete blocks are discarded.
6. Live blocks and i-nodes are copied into memory.
7. The old segment is marked free.
8. The live information is later written into a new segment at the front of the log.

12 Disk as a Circular Buffer

Circular Log View

LFS treats the disk like a circular buffer: the writer adds new segments at the front, while the cleaner frees old segments from the back.

- The writer thread appends new segments.
- The cleaner thread reclaims old segments.
- Live data from old segments is rewritten into newer segments.
- The i-node map must be updated when i-nodes move.
- This bookkeeping is complex but enables high write throughput.

13 Advantages and Costs

Point	Advantage	Cost
Write performance	Converts many small writes into large sequential writes	Requires buffering and segment management
Disk bandwidth	Can use nearly full disk bandwidth for segments	Depends on producing large enough segments
Reads	Many reads can be served from cache	Uncached reads may find blocks scattered in the log
Metadata	I-nodes can keep familiar structure	I-node locations require an i-node map
Space reuse	Cleaner reclaims obsolete blocks	Cleaning adds background work and bookkeeping

14 Key Points

- LFS is designed for systems where disk seeks are the bottleneck.
- It assumes many reads can be satisfied from cache.
- It improves small-write performance by writing large contiguous segments.
- It stores i-nodes and data blocks throughout the log.
- It uses an i-node map to locate the current copy of each i-node.
- It uses a cleaner to reclaim obsolete log segments.

Final Point

Log-structured file systems trade implementation complexity for faster write performance by turning many small random writes into large sequential log writes.